SYMBIOSIS INSTITUTE OF TECHNOLOGY, HYDERABAD
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

PROJECT REPORT
ON

"CINEPHILIA:
A Linked-List Based Personal Film Archive in C"

Subject: Data Structures - Lab

Submitted By:

Name	Branch	PRN
S. Dhruva Sai	CSE	25070721049
Md. Adnan	CSE	25070721037

INDEX

1. Acknowledgement
2. Abstract and Introduction
3. Methodology
4. Data Structures Used
5. Algorithms
6. Implementation and Testing
7. Applications and Future Scope
8. Screenshots of the Application

ACKNOWLEDGEMENT

We take this opportunity to express our heartfelt gratitude to everyone who has guided and supported us throughout the completion of our project titled *"CINEPHILIA: A Linked-List Based Personal Film Archive in C."*

First and foremost, we extend our deepest thanks to Dr. Rakesh Salakapuri, our professor and project guide, for his constant encouragement, valuable guidance, and continuous support. His insight into data structures and algorithms helped us choose the right structure for each part of the program and turn our idea into a complete, working application. His mentorship has been a constant source of inspiration throughout this journey.

We would like to express our sincere gratitude to Dr. Rajinikanth Aluvalu, Director of Symbiosis Institute of Technology, Hyderabad, for providing us with the necessary infrastructure, resources, and an environment that encourages innovation, experimentation, and research. His vision for academic excellence has motivated us to pursue this project with dedication and creativity.

Finally, we would like to express our gratitude to our peers and classmates for their helpful suggestions, cooperation, and moral support throughout the project. We are also thankful to Symbiosis Institute of Technology, Hyderabad, for providing us with the platform and resources to transform our ideas into reality.

ABSTRACT AND INTRODUCTION

Cinephilia is a terminal application written in C that serves as a personal archive of cinema. The user records the films they have watched or wish to watch — title, director, release year, genre, a personal rating and notes — keeps a **watchlist** and a dated **cinema journal**, receives **transparent recommendations**, and views **statistics** calculated from their own data.

Internally, the archive is held as a **doubly linked list** of films, ordered by a **merge sort adapted to linked lists**, while deletions can be reversed through an **undo stack**. Genre and director statistics are built with a **frequency table**, and film details fetched from The Movie Database (TMDB) are read by a **recursive-descent parser** that builds a **tree**. The archive is saved in a versioned plain-text format using **atomic writes**, so that it survives crashes and partial damage.

Most data structures are first met as isolated exercises. In a real program they must work together: a list that is searched, sorted, filtered, saved and restored without ever losing data or memory. Cinephilia was built to practise exactly that, inside an application we would want to use ourselves.

The project is written in standard C (gnu11) with no external libraries, compiles with Clang on macOS, and contains about **8,000 lines** of application code. Its correctness is checked by **1,367 automated unit checks** and **96 end-to-end checks** that drive the finished program.

Every recommendation is computed from the user's own ratings and states its reason; no artificial intelligence or online service is involved in it. Lookups on TMDB are optional, and the application works fully offline.

METHODOLOGY

1. **Requirement Analysis:**

- Identify what a film enthusiast needs: a collection, a watchlist, a journal of viewings, search, discovery and statistics.
- Require that data is never lost: saved after every change, readable after a crash, and private to the user.

2. **Data Modelling:**

- Each film is a **structure** holding its id, title, director, genre, notes, year, rating, status (watched or watchlist) and dates.
- Each viewing is a **journal entry** pointing to its film by id, so one film can be watched many times.

3. **Choice of Data Structures:**

- Match every operation to a structure: a **doubly linked list** for the archive, a **stack** for undo, a **frequency table** for statistics and a **tree** for TMDB replies.

4. **Algorithm Design:**

- **Merge sort** for ordering, **linear search** for substring matches, **shrunk averages** for recommendations, and **rejection** and **reservoir sampling** for fair random choices.

5. **Persistence:**

- A versioned text format with one record per line, saved atomically through a temporary file, with a backup of the previous version.

6. **Interface:**

- A keyboard-driven terminal interface using ANSI escape sequences, with an animated title, transitions and optional sound effects.

7. **Testing and Verification:**

- Unit tests for every module, end-to-end tests of the real program, and memory checks with Valgrind and AddressSanitizer.

8. **Tools:**

- This program has been developed in the C programming language (gnu11) using Clang, Make, the macOS Terminal, Git and GitHub, with the TMDB API for optional film details.

DATA STRUCTURES USED

Each data structure in Cinephilia was chosen because of the operations it must support, not merely to use it. The table below summarises them; each is then described in detail.

Data structure	Used for	Why it was chosen
Doubly linked list	The film archive	Films are added and removed one at a time; removal is O(1) and pointers stay valid
Stack	Undo delete	The most recent deletion must be undone first (LIFO)
Frequency table	Genre and director statistics	Counts and rating sums per key; few keys in a personal archive
Dynamic array	The cinema journal	Entries are appended and read in bulk
Tree (first-child / next-sibling)	TMDB replies (JSON)	Nested data with any number of children per node

4.1 Doubly Linked List

The archive is a **doubly linked list**: every film is allocated once on the heap and stores pointers to the previous and the next film. The archive keeps pointers to the **head** and the **tail** and a count of films.

```
typedef struct Film {
    unsigned id;
    char title[160];
    char director[96];
    char genre[48];
    int year, rating;
    FilmStatus status;          /* watched or watchlist */
    struct Film *prev;
    struct Film *next;
} Film;
```

To delete a film, its previous film is made to point past it and its next film to point back past it, updating the head or the tail at either end. An array would have to shift every later element instead. Because films never move in memory, a pointer to a film stays valid while others are added or removed.

Operation	Time complexity
Add a film (at the tail)	O(1)
Delete a film, once found	O(1)
Find a film by id	O(n)
Traverse all films	O(n)

4.2 Stack (Undo)

When a film is deleted, a record holding the film and all of its journal entries is **pushed** onto a stack. Choosing **Undo delete pops** the most recent record and restores the film with its original id. A stack fits because the user expects to recover the film deleted last — the **Last In, First Out** principle. The stack is a dynamic array of pointers that doubles its capacity when full, so push and pop take **O(1)** amortised time.

4.3 Frequency Table

Statistics and recommendations need, for every genre and director, the number of films and the sum of their ratings. A **frequency table** maps each key to these values. Keys are compared without regard to case, so “Drama” and “drama” count as one genre. Lookup is linear, **O(k)** for k distinct keys, which is fast enough for the few dozen genres of a personal archive; the table is then sorted by count.

4.4 Dynamic Array (Journal)

Journal entries are stored in a growable array that doubles in size when full. Each entry stores its own id, the id of its film, a date and a note, so a film may have any number of viewings.

4.5 Tree (First-Child / Next-Sibling)

Replies from TMDB are in JSON format, which nests objects inside arrays inside objects. They are parsed into a **tree** in which every node stores only two pointers: its **first child** and its **next sibling**. The children of a node therefore form a linked list, and a node can have any number of children.

```
object
├─ title     "In the Mood for Love"
├─ genres    array
│    ├─ object  name: "Drama"
│    └─ object  name: "Romance"
└─ credits   object
     └─ crew  array ...
```

ALGORITHMS

5.1 Merge Sort on a Linked List

The collection can be ordered by title, director, year, rating or date added. Sorting is done with **merge sort applied directly to the linked list**:

1. **Split:** find the middle of the list with a slow and a fast pointer, and cut it in two.
2. **Recurse:** sort each half in the same way.
3. **Merge:** join the two sorted halves, taking from the left half on a tie, so that equal films keep their order (the sort is **stable**).
4. **Repair:** one pass rebuilds every previous pointer and the tail.

```
Film *slow = list;
Film *fast = list->next;
while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
}
Film *second = slow->next;    /* start of the second half */
slow->next = NULL;            /* end of the first half */
```

Merge sort runs in **O(n log n)** time and needs no extra array. Quicksort is unsuitable here because it relies on random access, which a linked list does not provide. Titles are compared without a leading “The”, “A” or “An”, and films with an unknown year or no rating are always placed last.

5.2 Searching and Filtering

Search looks for the query anywhere inside a film’s title or director, ignoring case, so “mood” finds *In the Mood for Love*. A substring cannot be found with a sorted order or binary search, so a **linear search** is used, taking **O(n × m)** time for n films and a query of length m. Filters by genre, minimum rating, range of years and status are applied in the same pass.

5.3 Recommendation Scoring

Films on the watchlist are ranked by how highly the user has rated other films of the same genre and by the same director. To stop a single film from dominating, each average is pulled towards the user’s overall mean rating:

```
score = (sum of ratings + overall mean × 2) / (number of ratings + 2)
```

For example, with an overall mean of 7.0, one film rated 10 gives (10 + 14) / 3 = **8.0** rather than 10. Every recommendation is shown with its reason, for example: *“Wong Kar-wai — you average 9.5 across 2 of their films.”*

5.4 Fair Random Selection

Tonight's selection picks a film at random. A random number reduced with a plain modulo would favour some films slightly, so the program uses **rejection sampling**: numbers from the uneven remainder at the top of the range are discarded and drawn again. The generator (xorshift64*) can be given a fixed seed, which makes the tests repeatable. In a test of 10,000 picks from 10 films, every film was chosen within ±20% of the expected 1,000 times.

5.5 Double Feature (Reservoir Sampling)

A **double feature** pairs two films that belong together. Every pair is scored: 4 points for the same director, 2 for the same genre and 1 for the same decade. **Reservoir sampling** chooses uniformly among all pairs with the best score without storing them: the k-th best pair found replaces the current choice with probability 1/k.

5.6 Recursive-Descent Parsing

The JSON parser has one function per kind of value. **parse_value** calls **parse_container** for an object or array, which calls **parse_value** again for each element, so the call stack follows the nesting of the document. A **depth limit of 64** prevents a malicious reply from exhausting the stack, and any malformed input is rejected instead of crashing the program. Parsing takes **O(n)** time for n characters.

IMPLEMENTATION AND TESTING

6.1 Program Structure

The program is divided into modules, each with a single responsibility. Nothing below the screen layer prints to the terminal, so every data structure and algorithm can be tested on its own.

Layer	Files	Responsibility
Screens	main.c	Every screen and the order in which they appear
Interface	ui.c, wordmark.c, audio.c, settings.c	Drawing, keyboard input, animation, sound and preferences
Archive logic	film.c, search.c, discover.c, stats.c, tmdb.c	Films and journal, sorting, recommendations, statistics, TMDB
Foundations	storage.c, json.c, text.c, sb.c, dsa/stack.c, dsa/tally.c	Files, JSON, text handling and the reusable data structures

6.2 Persistence

The archive is a plain UTF-8 text file with a version header and one record per line: **F** lines are films and **J** lines are journal entries.

```
CINEPHILIA 1
F  1  w  2010  10  ...  Inception    Christopher Nolan  Action
J  1  1  2026-11-01  ...
```

- **Atomic saving:** the new archive is written to a temporary file, flushed to the disk, the old file is kept as a backup, and the temporary file is then renamed over it. An interrupted save leaves either the old archive or the new one, never a mixture.
- **Damage is contained:** a malformed line is skipped and counted while the rest of the archive loads, and the damaged original is copied before anything is rewritten.
- **Safety:** a file that is not a Cinephilia archive, or comes from a newer version, is opened read-only and never overwritten.

6.3 TMDB Integration

When the environment variable TMDB_API_KEY is set, the **Add a film** screen offers to look the film up on The Movie Database. Two requests are made — a search for the best match, then its details and credits — and the user confirms before the director, year and genre are used. The key is passed to the system's curl program through a pipe, so it never appears in the process list, the source code or the repository. A rejected key, a lost connection, an unknown title and TMDB's rate limit each produce a clear message.

This product uses the TMDB API but is not endorsed or certified by TMDB.

6.4 Terminal Interface

The interface is drawn with ANSI escape sequences and reads single key presses by switching the terminal to raw mode. Each screen is composed in memory and written at once, which prevents flicker. Signal handlers restore the terminal on exit, on Ctrl+C and around Ctrl+Z. Optional sound effects are original sounds synthesised by the program's own code and are played without blocking input.

6.5 Testing

Test	Result
Unit tests (text, list, sorting, storage, discovery, statistics, JSON, TMDB, audio)	1,367 checks, all passed
End-to-end tests of the real program	96 checks, all passed
Memory checks (Valgrind, AddressSanitizer)	No errors or leaks
Compilation with -Wall -Wextra -Wpedantic -Werror	No warnings

The end-to-end tests cover a first launch with no archive, adding, searching, editing and deleting films, reloading after a restart, random selection with zero, one and many films, invalid input, statistics, damaged files and failed saves.

APPLICATIONS AND FUTURE SCOPE

Applications

1. Keeping a personal, private record of films watched and films to watch.
2. Maintaining a dated journal of viewings and personal reflections.
3. Choosing what to watch next from one's own collection and ratings.
4. Understanding one's taste through genre, director and decade statistics.
5. Demonstrating linked lists, sorting, stacks, trees and sampling in a complete application.

Future Scope

- A **hash table** to look films up by title in O(1) average time instead of O(n).
- An **AVL tree** indexed by release year for fast range queries such as "films from 1960 to 1979".
- A **min-heap** to choose the top k recommendations in O(n log k) time.
- A **graph** connecting films that share a director or genre, with **breadth-first search** to show how two films are related.
- Searching TMDB for films outside the archive, and "more like this" suggestions for highly rated films.

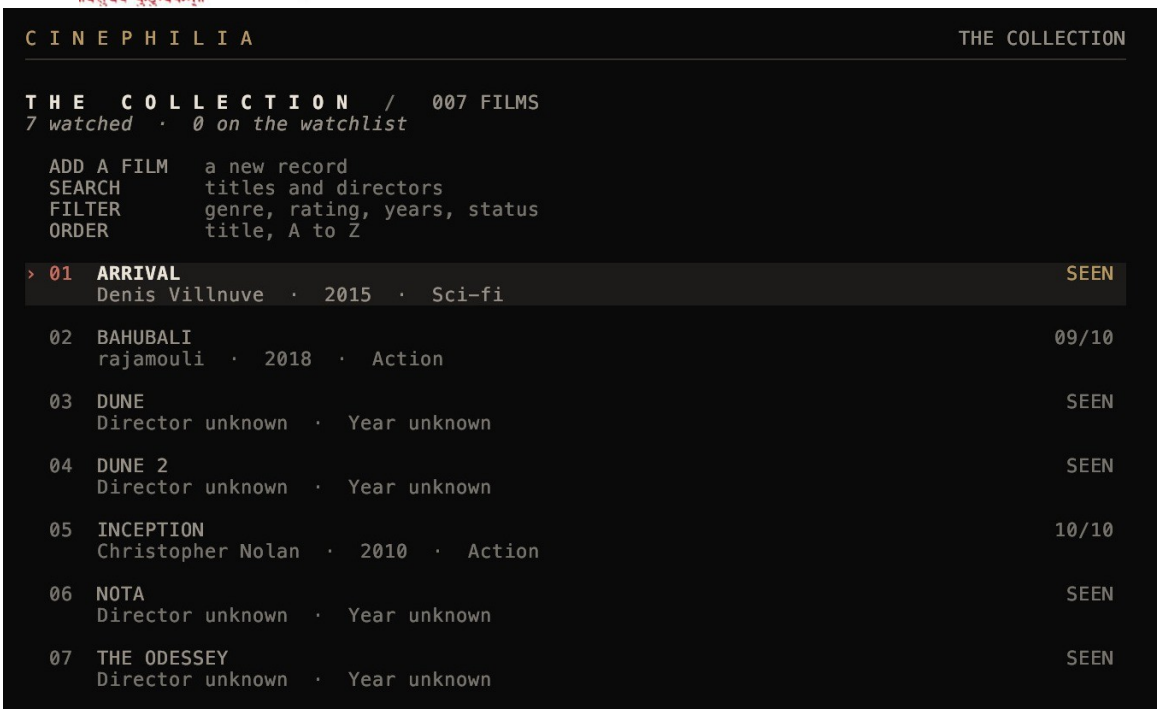
SCREENSHOTS OF THE APPLICATION



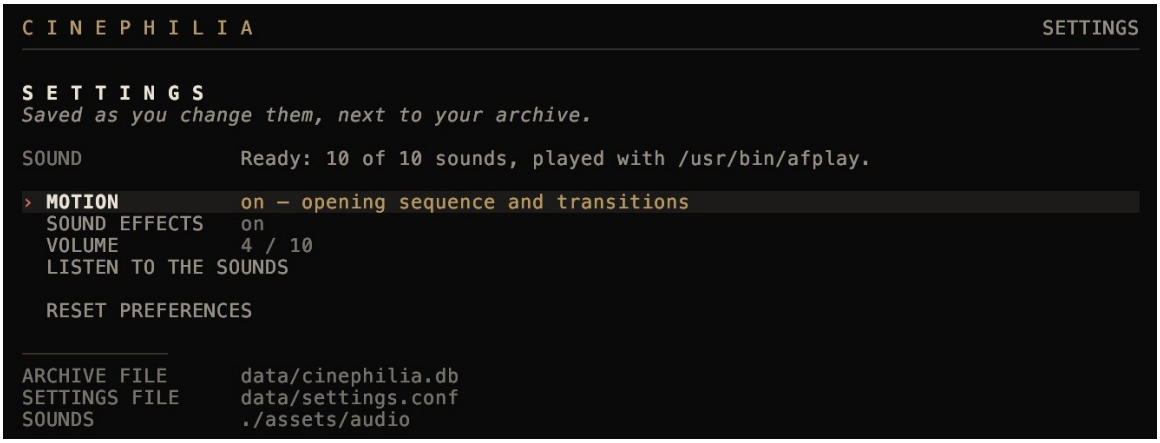
(this is the opening title card: the CINEPHILIA wordmark appears between two film strips, followed by the tagline)



(this is the main menu, the archival index, with its nine numbered sections)



(this is the collection: each film is a numbered catalogue entry with its director, year, genre and rating)



(this is the settings screen, where motion and sound can be turned on or off and the volume changed)



(these statistics are calculated from the stored films: the total, the average rating, the highest-rated films, genres, viewing history, decades and directors)